LOG8715 - Rapport de TP1

Groupe 14

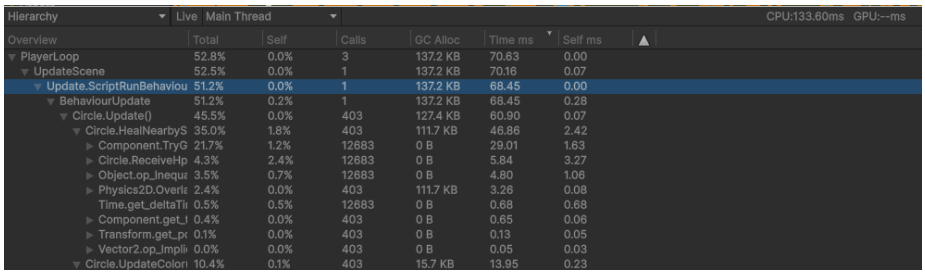
15 Mars 2026

1 Gestion de mémoire

Afin d'optimiser l'espace mémoire prise par des structures personnalisées, nous devons considérer l'effet du *padding* relié à l'alignement de données. *UnoptimisedStruct1* et *UnoptimizedStruct2* contiennent des types de taille fixe, tel *float* et *int*, mais aussi des types the taille variables comme *bool*, des pointers ou des types **enum**. Pour minimiser le padding, on organise les attributs par leur taille, tel *Vector3* > *double* > *int* > *float* > *FriendState* > *bool*. À la base, un type *enum* devrait être de type *int*, ou de taille 4, mais ça taille pourrait être réduite à un type *byte* ou augmenté. Dans ce cas, nous le laissons à ça taille originelle. Pour les pointeurs, leur taille peuvent varier entre 4 et 8 bytes, donc ils seront placés avant les types de tailles 4 comme *int* et *float*.

Originellement, le *memory profiler* estimait une mémoire occupée par *UnoptimisedStruct1* de 25.9 MB, puis 64.8 MB par *UnoptimizedStruct2*. Après l'organisation des attributs, la première valeur à réduit à 22.9 MB, la seconde inchangée. Le changement de *FriendState* à un type *byte* réduit ces valeurs à 14.0 MB et 56.5 MB. Le dernier changement, bien que risqué sur la maintenabilité, serait de réunir les types **bool** en un seul byte contenant alors des flags. Ce changement nécessiterait une modification du constructeur, qui n'est pas autorisé.

2 Utilisation du profileur CPU



Hierarchy | Live | Main Thread | CPU:133.60ms GPU:--ms

Overview	Total	Self	Calls	GC Alloc	Time ms	Self ms
PlayerLoop	52.8%	0.0%	3	137.2 KB	70.63	0.00
UpdateScene	52.5%	0.0%	1	137.2 KB	70.16	0.07
Update.ScriptRunBehaviou	51.2%	0.0%	1	137.2 KB	68.45	0.00
BehaviourUpdate	51.2%	0.2%	1	137.2 KB	68.45	0.28
Circle.Update()	45.5%	0.0%	403	127.4 KB	60.90	0.07
Circle.HealNearbyS	35.0%	1.8%	403	111.7 KB	46.86	2.42
Component.TryG	21.7%	1.2%	12683	0 B	29.01	1.63
Circle.ReceiveHp	4.3%	2.4%	12683	0 B	5.84	3.27
Object.op_Inequa	3.5%	0.7%	12683	0 B	4.80	1.06
Physics2D.Overla	2.4%	0.0%	403	111.7 KB	3.26	0.08
Time.get_deltaTi	0.5%	0.5%	12683	0 B	0.68	0.68
Component.get_t	0.4%	0.0%	403	0 B	0.65	0.06
Transform.get_p	0.1%	0.0%	403	0 B	0.13	0.05
Vector2.op_Impli	0.0%	0.0%	403	0 B	0.05	0.03
Circle.UpdateColori	10.4%	0.1%	403	15.7 KB	13.95	0.23

Figure 2.1: Premier profilage

Dans cette exercice on nous demande de profiler les trames de notre jeu pour les optimiser ensuite. En faisant un premier profilage on remarque que les pires frames peuvent monter 130 ms ce qui est très loin des 60 fps attendu et ce pour seulement 400 cercles au lieu des 8000 attendus (Fig 2.1).

On peut voir que circle.HealNearbyShape() représente 35% du budget alloué à la PlayerLoop ce qui est énorme (Fig 2.1), notre première optimisation portera donc sur ça. On remarque que cette méthode est appelée une fois par cercle par frame, en effet on cherche tous les colliders, on leur alloue un tableau et on vérifie les composants. Or on peut se passer de cette allocation

on va donc temporairement changer l'appelle de Physics2D.OverlapCircleAll à Physics2D.OverlapCircleNonAlloc.

Ensuite on peut voir que Object.FindFirstObjectByType représente 9% du budget alloué et est appelé autant de fois que de cercles puisque cet appel se fait lors des updates de couleurs pour chaque cercle et sur toute la grille. C'est donc extrêmement redondant et on peut stocker cette grille de manière globale plutôt que de la chercher de nouveau en continue. On effectue cela dans Start() et ainsi on n'a plus qu'à lire les valeurs voulues dans UpdateColor(). On fait de même avec SpriteRender qui était appelé une fois par cercle aussi.

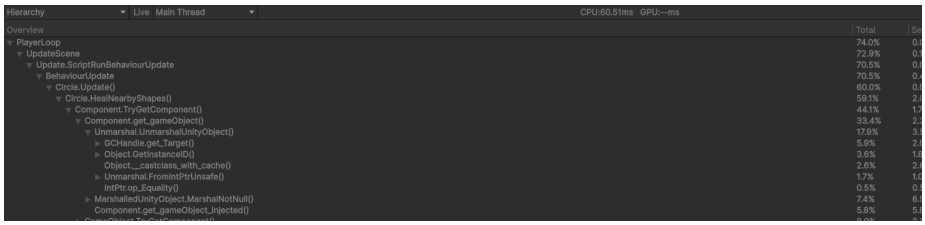


Figure 2.2: Deuxième profilage

On effectue maintenant de nouveau un profilage pour voir comment à évoluer notre problème mais avec cette fois 8000 cercles. On obtiens dorénavant des pires frames à 60 ms ce qui prouve que notre optimisation est en bonne voie. On constate que dorénavant Circle.Update() prend 60% du budget malgrès les changements de non-allocations. Le réel problème est donc en fait à l'appel du physics engine. En effet les cercles sont sur une grille fixe donc chaque cercle connaît sa position dans la grille, l'utilisation d'un moteur physique pour déterminer les voisins est donc superflue. On peut donc modifier la méthode HealNearbyShapes() par un calcul simple des voisins ((i-1, j), (i+1, j), (i, j-1) et (i, j+1)).

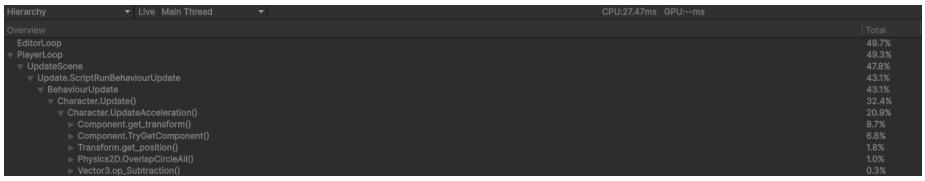


Figure 2.3: Dernier profilage

On obtiens donc finalement des frames au pire cas à 24 ms (en moyenne à beaucoup moins) ce qui est idéal pour faire tourner notre jeu à 60 fps !

3 Parallélisme

3.1 Composantes

Plantes: Lifetime, Decay, Position, Taille
Proies et Prédateurs: Lifetime, Decay, Position, Vitesse, Reproduction

Puisque les proies et prédateurs ont les mêmes attributs, ils seront réunit sous un même archétype **acteur**. Les systèmes sont ceux de Spawn/Respawn, Mouvement, Dégradation et Taille (pour les plantes). Les jobs associés sont eux de déterminer la cible la plus proche, les collisions affectant par la suite le vieillissement, la nouvelle taille des plantes puis la génération de nouvelle position au respawn.